RAGHU ENGINEERING COLLEGE

(AUTONOMOUS)
Affiliated to JNTU GURAJADA, VIZIANAGARAM
Approved by AICTE, Accredited by NBA, Accredited by NAAC with A+ grade
www.raghuengineering.com
2022-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that this project entitled **"Automated Kidney Stone Detection Using Image Processing Techniques"** done by "**VADAMA SURENDRA (RollNo. 22981A44I7), TALLURI KRISHNA SANJAY (RollNo. 22981A44H8), SHAIK NAGOOR SHARIF (RollNo.21985A4203 22981A44H1),AYINAVELLI UDAYKIRAN (RollNo. 23985A4419) "** are students of B.Tech in the Department of Computer Science and Engineering, Raghu Engineering College, during the period 2022-2026, in partial fulfilment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering(DATA SCIENCE) to the Jawaharlal Nehru Technological University, Gurajada, Vizianagaram is a record of bonafide work carried out under my guidance and supervision.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree.

Internal Guide	Head of the Department
MR.P.SRINIVASU	Dr. K .V. Satyanarayana
Dept of CSE(DS),	Dept of CSE(DS),
Raghu Engineering College,	Raghu Engineering College,
Dakamarri,	Dakamarri,
Visakhapatanam	Visakhapatanam

EXTERNAL EXAMINER

DISSERTATION APPROVAL SHEET

This is to certify that the dissertation titled

Automated Kidney Stone Detection Using Image Processing Techniques

BY

VADAMA SURENDRA	**(**ROLL NO. 22981A44I7**)**
TALLURI KRISHNA SANJAY	**(**ROLLNO. 22981A44H8**)**
SHAIK NAGOOR SHARI	**(**ROLL NO. 22981A44H1**)**
AYINAVELLI UDAYKIRAN	**(**ROLL NO. 23985A4419**)**

is approved for the degree of ***Bachelor of Technology***

Mr.P.SRINIVASU

Internal Examiner

External Examiner

HOD

Date:

DECLARATION

This is to certify that this project titled **"Automated Kidney Stone Detection Using Image Processing Techniques"** is bonafide work done by us, in partial fulfilment of the requirements for the award of the degree B.Tech and submitted to the **Department of Computer Science and Engineering (DS), Raghu Engineering College, Dakamarri.**

We also declare that this project is a result of our effort and that has not been copied from anyone and we have taken only citations from the sources which are mentioned in the references. This work was not submitted earlier at any other University or Institute for the reward of any degree.

VADAMA SURENDRA
(ROLLNO. 22981A44I7)

T. KRISHNA SANJAY
(ROLLNO. 22981A44H8)

SK. NAGOOR SHARIF
(ROLL NO. 22981A44H1)

A. UDAYKIRAN
(ROLLNO. 23985A4419)

Date:

Place:

ACKNOWLEDGEMENT

We express our sincere gratitude to my esteemed Institute "Raghu Engineering College",which has provided us with an opportunity to fulfil the most cherished desire to reach my goal.

We take this opportunity with great pleasure to put on record our ineffable personal indebtedness to **Sri Raghu Kalidindi, Chairman of Raghu Engineering College** for providingnecessary departmental facilities.

We would like to thank the Principal **Dr.Ch.Srinivasu, Dr A.Vijay Kumar**- Dean planning & Development, **Dr E.V.V.Ramanamurthy**-Controller of Examinations, and the Management of **"Raghu Engineering College",** for providing the requisite facilities to carry them out the project in the campus.

Our sincere thanks to **Sri S.Srinadh Raju**, Dean, Department of Computer Science and Engineering, Raghu Engineering College, for his kind support in the successful completion of this work.

Our sincere thanks to **Dr.K.V.Satyanarayana**, Program Head, Department of Computer Science and Engineering, Raghu Engineering College, for this kind support in the successful completion of this work.

We sincerely express our deep sense of gratitude to **P.SRINIVASU, Assistant Professor** Department of Computer Science and Engineering, Raghu Engineering College, for her perspicacity, wisdom, and sagacity coupled with compassion and patience. It is our great pleasure to submit this work under her wing.

We extend deep-hearted thanks to all faculty members of the Computer Science department for the value-based imparting of theory and practical subjects, which were used in the project.

We are thankful to the non-teaching staff of the Department of Computer Science and Engineering, Raghu Engineering College, and our parents for their inexpressible support.

Regards,

VADAMA SURENDRA (22981A44I7)
TALLURI KRISHNA SANJAY(22981A44H8)
SHAIK NAGOOR SHARI (22981A44H1)
AYINAVELLI UDAYKIRAN(23985A4419)

ABSTRACT

Kidney stones are now a serious issue that, if not treated right away, can lead to complications and occasionally even require surgery to remove. Image processing can greatly increase the likelihood of accurately detecting stones well in advance. It has the propensity to produce accurate results and is an automatic method of finding the stone. Because kidney stones have low contrast and speckle noise, finding them via ultrasound imaging is a very difficult task. By using appropriate Image processing-MatLab approaches, this problem is solved. The speckle noise in the ultrasound image is first pre-processed using the image restoration technique. Stone placement detection was then performed using the reconstructed image.

TABLE OF CONTENTS

CHAPTER	Page No
1. Introduction	1
1.1 Introduction to Image processing	1
1.2 Applications	2
1.3 Introduction to Kidney Stone Detection in Ultra-Sound Scans Using Image processing	3
2. Literature Survey	4
3. Requirements and Analysis	6
3.1 Existing System	6
3.2 Proposed System	6
3.2.1 Project Scope	6
3.2.2 Project Objectives	6
3.3 Functional Requirements	6
3.4 Non Functional Requirements	7
3.4.1 Hardware Specifications	7
3.4.2 Software Specifications	7
3.5 Analysis	8
3.5.1 Entity Objects	9
3.5.2 Boundary Objects	9
3.5.3 Control Objects	9
3.5.4 Object Interaction	9
3.5.5 Object Behaviour	9
4. System Design	11
4.1 Design Goals	11
4.2 System Architecture	12
4.3 System Design	12
4.4 UML Diagrams	12
4.4.1 Usecase Diagrams	13
4.4.2 Class Diagrams	14
4.4.3 Activity Diagrams	15

4.4.4 Sequence Diagrams 16
4.5 Data Flow Diagrams 17
4.6 Flow Chart 18
5. Implementation 20
5.1 Coding Approach 20
5.2 Information Handling 20
5.3 Programming Style 21
5.4 Verification and Validation 21
5.5 Implementation of Project 21
5.5.1 Algorithmic steps 30
6. Testing 33
6.1 Testing Activities 33
6.2 Testing Types 33
7. Results 38
7.1 Start Page 38
7.2 Image Selected with Stone 38
7.3 Stone Detection 39
7.4 Size of Detected Stone 39
7.5 Image Selected without Stone 40
7.6 Stone Detection 40
7.7 Sizes of Detected Stone 41
8. Sample Code 42
8.1 Code for application 42
8.2 Code for GUI 45
9. Conclusion 60
10. References 61

LIST OF TABLES

Title	Page No
6.3 Test cases	36

LIST OF FIGURES

Title	Page No
3.4.1.1 MATLAB Logo	7
4.3.1 System's Block Diagram	11
4.4.1.1 Use Case Diagram	13
4.4.2.1 Class Diagram	14
4.4.3.1 Activity Diagram	15
4.4.4.1 Sequence Diagram	16
4.5.1 DFD	17
4.6.1 Flow chart of the system	18
7.1.1 Start Window	38
7.2.1 Image Selected with Stone	38
7.3.1 Stone Detected	39
7.4.1 Size of Detected Stone	39
7.5.1 Image Selected without Stone	40
7.6.1 Size of Detected Stone	40
7.7.1 Size of Detected Stone	41

INTRODUCTION

1. INTRODUCTION

1.1 Introduction to Image processing

Image processing is a domain of study that involves the analysis, manipulation, and interpretation of digital images using various algorithms and mathematical techniques. It is a multidisciplinary field that combines aspects of computer science, mathematics, physics, and engineering. The goal of Image processing is to extract useful information from digital images, improve image quality, or perform various image-based tasks, such as object detection and recognition, segmentation, registration, and restoration.

Image processing can be broadly classified into two categories: analog Image processing and digital Image processing. Analog Image processing involves the manipulation of physical images using optical and electrical techniques, whereas digital Image processing involves the manipulation of digital images using computational algorithms. Image processing finds applications in a wide range of fields, including medical imaging, surveillance and security, remote sensing, robotics and automation, entertainment, and agriculture. In the medical field, Image processing is used to diagnose diseases, plan treatments, and monitor the progress of medical procedures. In the field of surveillance and security, Image processing is used to detect and track objects, recognize faces, and identify potential threats. Some common techniques used in Image processing include filtering, segmentation, feature extraction, and pattern recognition. Filtering techniques are used to remove noise or enhance certain features of an image, while segmentation techniques are used to partition an image into regions of interest. Feature extraction techniques are used to extract relevant information from an image, while pattern recognition techniques are used to recognize and classify objects in an image. In summary, Image processing is an interdisciplinary field that involves the use of mathematical algorithms and computational techniques to analyze, manipulate, and interpret digital images. It has numerous applications in various domains, and a wide range of techniques and tools are used in Image processing to extract useful information from images, improve their visual quality, and perform image-based tasks.

1.2 Applications

- **Medical imaging**: Image processing plays a crucial role in the field of medical imaging. Images of the human body can be analysed to diagnose diseases and plan treatments. Techniques such as image segmentation and registration are used to extract information from medical images, while image enhancement techniques are used to improve the quality of the images.
- **Surveillance and security**: Image processing is used in surveillance and security systems to detect and track objects, recognize faces, and identify potential threats. Techniques such as object detection, tracking, and classification are used in these applications.
- **Remote sensing**: Image processing is used in remote sensing to extract information from satellite images and aerial photographs. This information can be used to monitor environmental changes, detect natural disasters, and perform urban planning.
- **Robotics and automation**: Image processing is used in robotics and automation to enable robots and automated systems to navigate and perform tasks. Techniques such as object recognition and localization are used in these applications.
- **Entertainment**: Image processing is used in the entertainment industry to create special effects and enhance visual quality in movies, television shows, and video games. Techniques such as image compositing, motion tracking, and 3D rendering are used in these applications.
- **Agriculture**: Image processing is used in agriculture to monitor crops, detect diseases, and optimize irrigation and fertilization. Techniques such as image segmentation and pattern recognition are used in these applications.
- **Biometrics**: Image processing is used in biometrics to identify individuals based on their physical or behavioural characteristics. Techniques such as face recognition, fingerprint recognition, and iris recognition are used in these applications.

1.3 Introduction to Kidney Stone Detection in Ultra-Sound Scans using Image processing

Millions of individuals all around the world struggle with kidney stones, which is a frequent health issue. These are tiny, hard mineral and salt deposits that develop in the kidneys and can be quite painful and uncomfortable as they move through the urinary tract. Using conventional techniques to find kidney stones, such as CT scans and X-rays can be expensive and subject patients to ionizing radiation. Non-ionizing radiation, which is thought to be safer, is used in MRI scans. but very pricey for the average person to afford.

Ultrasonic (US) scans, on the other hand, don't involve ionizing radiation and are safe for routine use. Ultrasound scans are less expensive than other scanning techniques, but they are not typically advised for the detection of stones because they are not very accurate in detecting tiny stones due to the disturbance in them, we can't expect accurate results.

We present a novel approach for detecting kidney stones using Image processing techniques in MATLAB. The proposed system takes US Scan images of the Kidney and applies various Image processing techniques such as thresholding, median filtering, and connected component analysis to detect and locate the stones in the kidney. The result of the process is an output image with red boxes around the detected stones, along with the size of each stone. As the proposed system works on US scans and the usage of system should be user-friendly the working system is integrated with a graphical user interface which is implemented using MATLAB's graphical user interface (GUI) tool. This proposed system will make the kidney stone diagnosis process simpler, safer and also affordable in terms of cost to many patients who are in need of diagnosis.

LITERATURE SURVEY

2. LITERATURE SURVEY

Tsao, Chang and Lin, published a research paper named after **"Ultrasonic Renal-Stone Detection and Identification for Extracorporeal Lithotripsy"** analysed the exact position of palpable urinary calculus and demonstrated its problems which are crucial for extracorporeal shock wave lithotripsy. Because it constantly uses stun waves to detect kidney stones. But the miss-hit of shock waves may harm the tissue badly. Their investigation revealed that the spot clamour exists in all ultrasonic images that should be removed. [1]

Viswanath and Gunasundari, published a research paper named after **"Design and analysis performance of kidney stone detection from ultrasound image by level set segmentation and ANN classification"** by lowering specific vitality levels that confirm the presence of urinary calculus in a specific area, improved accuracy in 2014. To complete the procedure, the artificial neural network idea was effectively applied. But the system they proposed will not specify the size of the stones detected.[2]

Angshuman Khan, **Rupayan Da** and **M C Parameshwara**, published a research paper named after "**Detection of kidney stone using digital Image processing: a holistic approach"** finds whether the kidney scan image has stones in it or not but it doesn't detect and calculate the size of the stone. [3]

Sakshi Garg, Angadpreet Walia, Abhilasha Singh & Anju Mishra , published a research paper named after "**A Study on Shape Detection: An Unexplored Parameter in the Gallstones Identification** the system uses several Image processing methods to detect stones in Gall Bladder. [4]

A renal ultrasound is a very useful exam which allows doctors to quickly evaluate the condition of kidneys without exposing patients to radiation or using IV contract. The test can be repeated without any harm. Still, it cannot reveal every type of kidney stone, therefore patients usually require additional exams [5].

F. L. Coe, A. Evan, and E. Worcester, in article [6] "**Kidney stone disease**," the study developed a method for defining the boundary of regions of interest in a digital KUB CT scan but it doesn't detect the stone.

V. Romero, H. Akpinar, and D. G. Assimos, [7] developed a method for detecting the object of interest (kidney stones) including its size and location in a digital KUB CT scan image.

R. C. Gonzalez and R. E. Woods, [8] book on Image processing techniques applied in the program include Contrast Adjustment, Segmentation.

A. C. Kak and A.Rosenfeld [9] book in the area of Digital Picture Processing.

Despite numerous studies on kidney stone detection, some recent works which are done by **Mehmet Baygin , Orhan Yaman, Prabal Datta Barua** [10] and Tanya Borges, **Akash Rai, Dharm Raj, Danish Ather, Keshav Gupta** [11] still face certain limitations. Some of these include lack of focus on reducing unwanted signals from ultrasound images and improving image quality, weak segmentation in detecting stones, inaccurate detection and location of stones, and cumbersome execution time and memory usage.

REQUIREMENTS AND ANALYSIS

3. REQUIREMENTS AND ANALYSIS

3.1 Existing system

The existing process of detecting Kidney stones is done using CT and MRI scans which are cost effective and harmful for human body. Ultrasound Scans are an alternative less cost-effective process and doesn't involve any harmful process like ionization during the scan. But this method of diagnosis is not usually preferred by doctors as US scans cannot detect stones with size lesser than 2 to 3 mm due to denser noise in the scan image.

3.2 Proposed system

The proposed system can detect stones lesser than 3mm which breaks the limitation of US scans in general. The system also outputs the image of scan with positions of the stones located with red boxes around them and the size if the stones will also be displayed. The output is displayed to the user through a GUI which makes this user-friendly.

3.2.1 Project scope

The scope of the project is to reduce the cost of kidney stone diagnosis by replacing MRI and CT scans which are highly cost effective with Ultra-Sound scans. Which makes the system more reliable.

3.2.2 Project objectives

The system has some objectives to achieve. They are as follows

- To remove speckle noise from Ultra-Sound scans and reconstruct them to be able to use for detection of smaller stones.
- To detect the stones in the kidney and project the precise locations of the stones and their sizes.

3.3 Functional Requirements

The functional requirements describe the inputs and outputs of the application. The functional requirements of this project are as follows:

- **Image acquisition**: The system should be able to acquire high-quality images of the kidneys through ultrasound scans.
- **Image pre-processing**: The acquired images need to be pre-processed to remove noise, enhance contrast, and adjust brightness to ensure accurate detection of kidney stones.
- **Detection methods**: The system should have methods that can accurately detect and localize kidney stones in the pre-processed images based on their shape, size, and texture.
- **Size measurement**: The system should be able to measure the size of each detected kidney stone accurately.
- **User interface**: The system should have a user-friendly interface to enable easy uploading of images and displaying the output results.

3.4. Non-Functional Requirements:

- Accuracy
- Reliability
- Performance
- Interoperability

3.4.1 Hardware Specifications

- **Processor:** Any Intel or AMD x86–64 processor
- **RAM:** Minimum: 4 GB
- **Graphics:** No specific graphics card is required, but a hardware accelerated graphics card supporting OpenGL 3.3 with 1GB GPU memory is recommended.
- **Storage:** Minimum 8.0 GB for MATLAB and for installation of Image processing, Open CV and Digital Image processing Modules.

3.4.2 Software Specifications

- **Software:** MATLAB R2018a Version 9.4 or above.
- **Operating System:** Windows 10 (64- bit) or above.

MATLAB

MATLAB is a popular software tool used in Image processing that offers a range of functions and tools for analysing and manipulating digital images. It provides a variety

of built-in functions and toolboxes specifically designed for Image processing, including image filtering, feature extraction, image segmentation, and object detection.

MATLAB's Image processing capabilities include features such as image display, processing, and analysis. It provides a wide range of Image processing functions and algorithms, which can be used to perform tasks such as noise reduction, edge detection, image enhancement, and feature extraction. It also includes tools for morphological operations, image segmentation, and image registration.

MATLAB's Image processing capabilities are widely used in a variety of fields, including medical imaging, surveillance, and remote sensing. With its user-friendly interface, extensive documentation, and a large community of users, MATLAB is a popular choice for researchers, engineers, and professionals who require powerful and flexible Image processing capabilities.



Fig.3.4.1.1 MATLAB Logo

3.5 Analysis:

The analysis phase defines the requirements of the system, independent of how these requirements will be accomplished. This phase defines the problem that the customer is trying to solve. The deliverable result at the end of this phase is a requirement document. Analysis object model is represented by class and object diagrams. Analysis focuses on producing a model of the system, called the Analysis model, which is correct, complete, consistent, and verifiable. The analysis model is composed of three individual models: The Functional Model represented by use cases and scenarios, the Analysis Object Model, represented by class and object diagrams, and the Dynamic Model, represented by state chart and sequence diagrams. The analysis model represents the system under development from the user’s point of view. The analysis

object model is a part of the analysis and focuses on the individual concepts that are manipulated by the system, their properties and their relationships.

3.5.1 Entity Objects

The Analysis object model consists of entity, boundary and control objects. Entity objects represent the persistent information tracked by the system. Participating objects form the basis of the analysis model.

3.5.2 Boundary Objects

Boundary object is the object used for interaction between the user and the system. Moreover, it is an interface used to communicate with the system. Boundary objects represent the system interface with the actors. In each use case, each actor interacts with at least one boundary object. The boundary object collects the information from the actor and translates into an interface model from that can be used by the objects and by the control objects.

3.5.3 Control Objects

Control objects are responsible for coordinating entity objects and boundary objects. A control object is creating at the beginning of the use cases and ceases to exist at its end. Control objects usually do not have a concrete counterpart in the real world. Control object is a responsible for collecting information from the boundary objects and dispatching it to entity object. Here only authorized users can operate, and details obtained are verified. The control objects in this project are admission, result, placement, company and eligibility details.

3.5.4 Object Interaction

Interaction diagrams model the behaviour of use cases by describing the way groups of objects interact to complete the task. The two kinds of interaction diagrams are sequence and collaboration diagrams. Sequence diagrams generally show the sequence of events that occur.

3.5.5 Object Behaviour

State chart diagrams are used to describe the behaviour of a system. They define different states of an object during its lifetime. Each diagram usually represents objects

of a single class and tracks the different states of its objects through the system and these states are changed by events. State chart diagram describes the flow of control from one state to another state. State chart diagrams are very important for describing the states. States can be identified as the condition of objects when an event occurs.

SYSTEM DESIGN

4. SYSTEM DESIGN

System Design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. In System design, developers:

- Define design goals of the project
- Decompose the system into smaller sub systems
- Design hardware/software strategies
- Design persistent data management strategies
- Design global control flow strategies
- Design access control policies and
- Design strategies for handling boundary conditions.

System design is not algorithmic. It is decomposed of several activities. They are:

- Identify Design Goals
- Design the initial subsystem decomposition
- Refine the subsystem decomposition to address the design goals.

System Design is the transform of analysis model into a system design model. Developers define the design goals of the project and decompose the system into smaller subsystems that can be realized by individual teams. Developers also select strategies for building the system, such as the hardware/software platform on which the system will run, the persistent data management strategy, the goal control flow the access control policy and the handling of boundary conditions. The result of the system design is model that includes a clear description of each of these strategies, subsystem decomposition, and a UML deployment diagram representing the hardware/software mapping of the system.

4.1 Design Goals

Design goals are the qualities that the system should focus on. Many design goals can be inferred from the non-functional requirements or from the application domain.

User friendly: The system is user friendly because it is easy to use and understand.

Reliability: Proper checks are there for any failure in the system if they exist.

4.2 System Architecture

As the complexity of systems increases, the specification of the system decomposition is critical. Moreover, subsystem decomposition is constantly revised whenever new issues are addressed. Subsystems are merged into alone subsystem, a complex subsystem is split into parts, and some subsystems are added to take care of new functionality. The first iterations over the subsystem decomposition can introduce drastic changes in the system design model.

4.3 System Design

The overall project aim is to pre-process the input image of US scan of a kidney using Image processing filters available in MATLAB and use the reconstructed image to detect the stone locations and sizes of the stones detected to the user via the GUI through which the user can access the functionalities provided by the system.

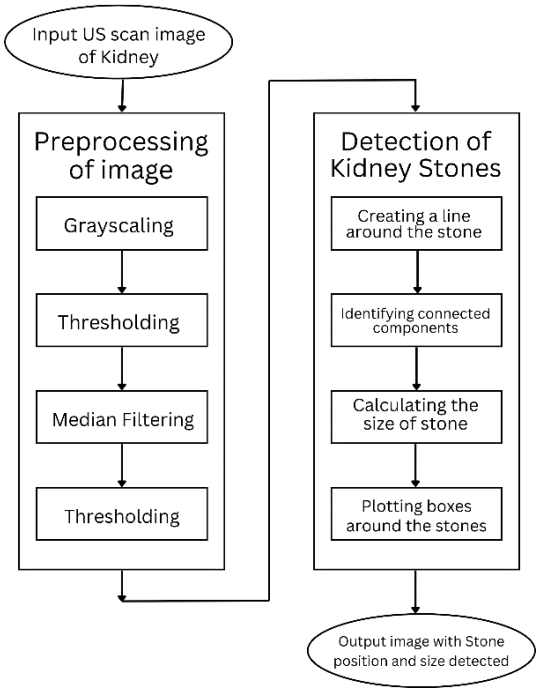


Fig. 4.3.1 System's Block Diagram

4.4 UML Diagrams

A UML diagram is a diagram based on the UML (Unified Modelling Language) with the purpose of visually representing a system along with its main actors, roles, actions,

artifacts or classes, in order to better understand, alter, maintain, or document information about the system.

UML is a modern approach to modelling and documenting software. In fact, it's one of the most popular business process modelling techniques. It is based on diagrammatic representations of software components. As the old proverb says: "a picture is worth a thousand words". By using visual representations, we are able to better understand possible flaws or errors in software or business processes.

Mainly UML has been used as a general-purpose modelling language in the field of software engineering. However, it has row found its way into the documentation of several business processes or workflows. For example, activity diagrams, a type of UML diagram, can be used as a replacement for flowcharts. They provide both a more standardized way of modelling workflows as well as a wider range of features to improves readability.

4.4.1 Use Case Diagram

In the Unified Modelling Language (UML), a use case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system. To build one, use a set of specialized symbols and connectors. An effective use case diagram can help the team discuss and represent:

- Scenarios in which system or application interacts with people, organizations, or external systems.
- Goals that your system or application helps those entities (known as actors) achieve.
- The scope of the system.

UML use case diagrams are ideal for:

- Representing the goals of system-user interactions.
- Defining and organizing functional requirements in a system.
- Specifying the context and requirements of a system.
- Modelling the basic flow of events in a use case.

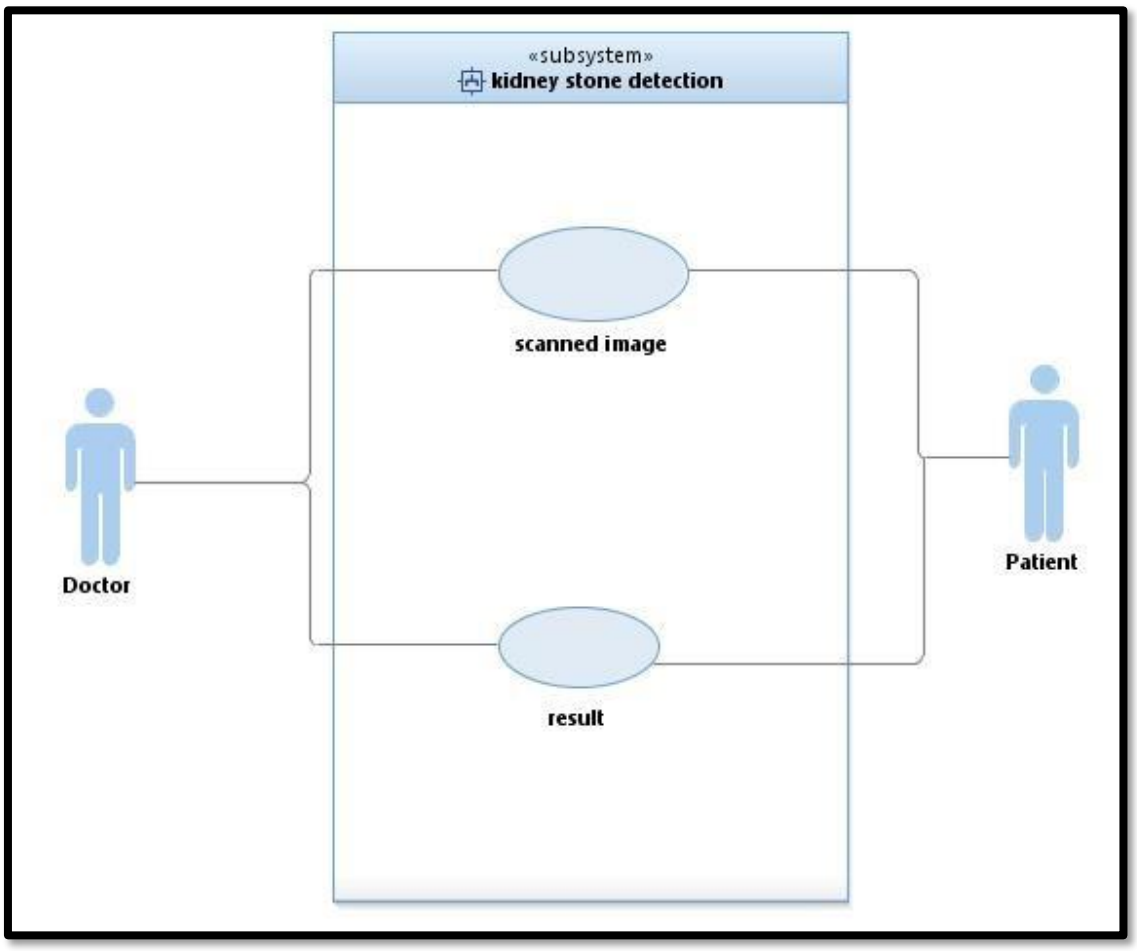


Fig. 4.4.1.1 Use Case Diagram

4.4.2 Class Diagram

Class diagrams are one of the most useful types of diagrams in UML as they clearly map out the structure of a particular system by modelling its classes, attributes, operations, and relationships between objects. It is a static diagram that represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.

The class shape itself consists of a rectangle with three rows. The top row contains the name of the class, the middle row contains the attributes of the class, and the bottom section express the methods or operations that the class may use. Classes and subclasses are grouped together to show the static relationship between each object.

The purpose of the class diagram can be summarized as:

- Analysis and design of the static view of an application.
- Describe responsibilities of a system.
- Base for component and deployment diagrams.
- Forward and reverse engineering.

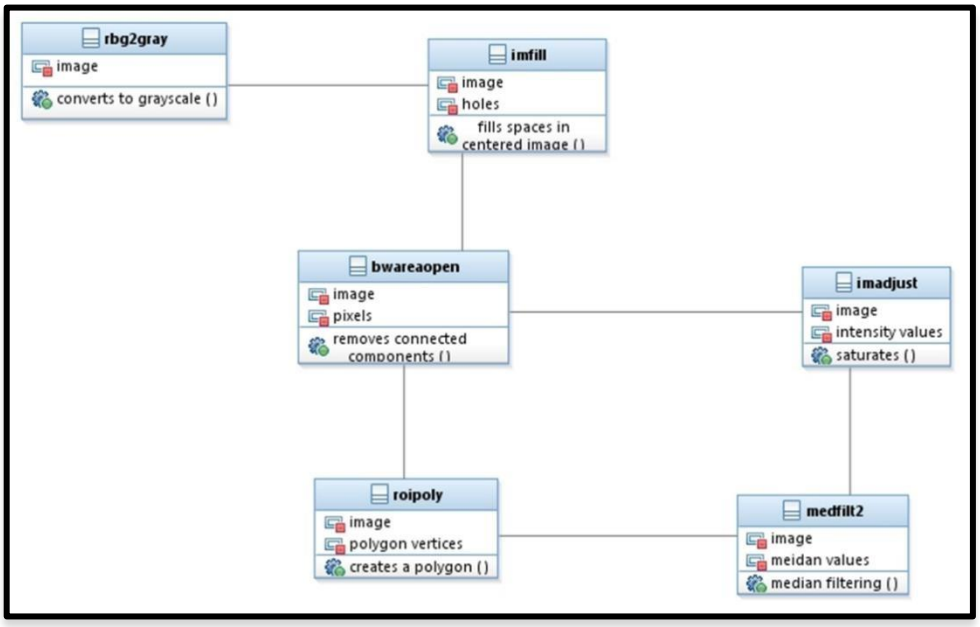


Fig. 4.4.2.1 Class Diagram

4.4.3 Activity Diagram

Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.

The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc. Activity is a particular operation of the system. Activity diagrams are not only used for visualizing the dynamic nature of a system, but they are also used to construct the executable system by using forward and reverse engineering techniques.

The purpose of an activity diagram can be described as

- Draw the activity flow of a system.
- Describe the sequence from one activity to another.
- Describe the parallel, branched and concurrent flow of the system.

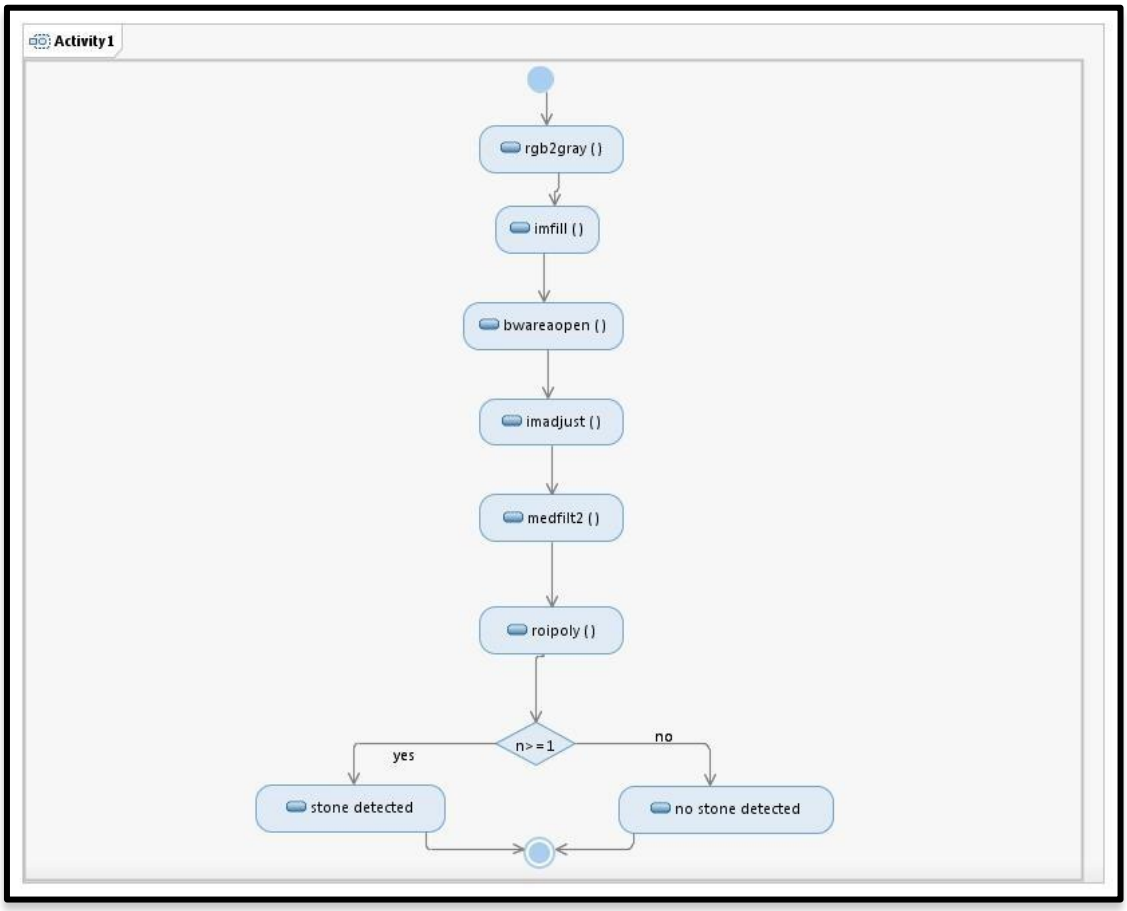


Fig. 4.4.3.1 Activity Diagram

4.4.4 Sequence Diagram

A sequence diagram is the most commonly used interaction diagram. It simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place. We can also use the terms event diagrams or event scenarios to refer to a sequence diagram. Sequence diagrams describe how and in what order the objects in a system function. These diagrams are widely used by businessmen and software developers to document and understand requirements for new and existing systems.

Sequence diagrams can be useful references for businesses and other organizations. Purpose of sequence diagrams are:

- Represent the details of a UML use case.
- Model the logic of a sophisticated procedure, function, or operation.

- See how objects and components interact with each other to complete a process.
- Plan and understand the detailed functionality of an existing or future scenario.

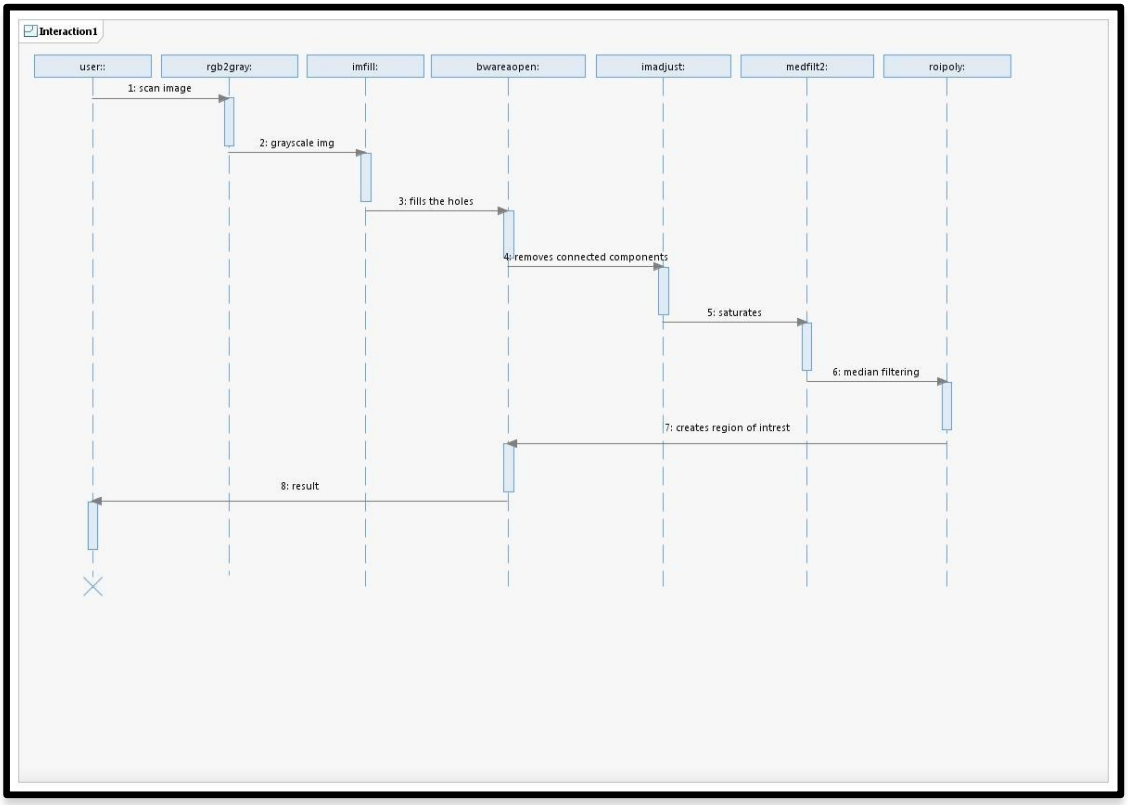


Fig. 4.4.4.1 Sequence Diagram

4.5 Data Flow Diagrams

A Data Flow Diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination. Data flowcharts can range from simple, even hand-drawn process overviews, to in-depth, multi-level DFDs that dig progressively deeper into how the data is handled.

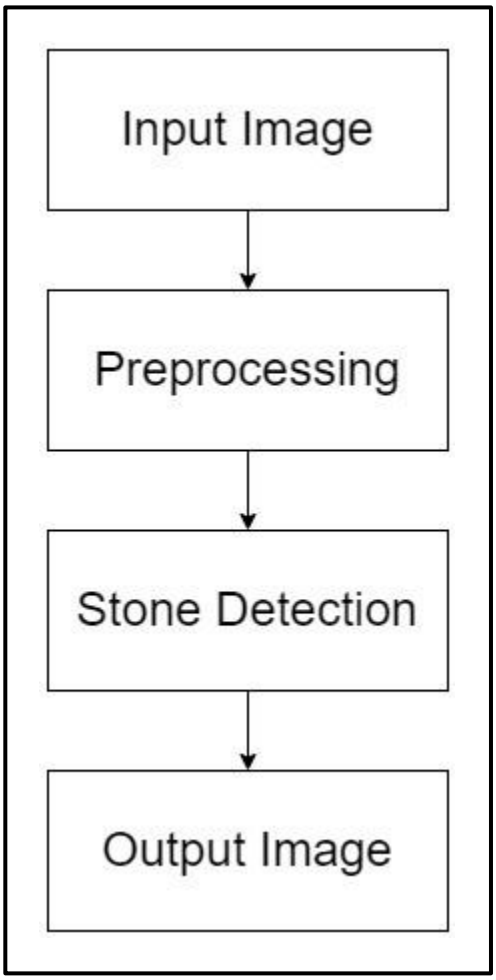


Fig 4.5.1 DFD

4.6 Flow Chart

A flowchart is a type of diagram that represents an algorithm, workflow or process. Flowchart can also be define as a diagrammatic representation of an algorithm. The flowchart shows the steps as boxes of various kinds, and their order by connecting the boxes with arrows. This diagrammatic representation illustrates a solution model to a given problem. Flowcharts are used in analysing, designing, documenting or managing a process or program in various fields.

The first step in flow of this system involves in accepting US scans as input to the system and performs the listed 2 phases of the system to achieve the result of the proposed system.

The system at the beginning performs pre-processing on the input image using various filters and methods pre-defined in MATLAB and eliminates the speckle noise from the scan image and reconstructs the image into a desired format for stone size and location detection if the value of n is greater then 1. The procedure is represented in the below flowchart.

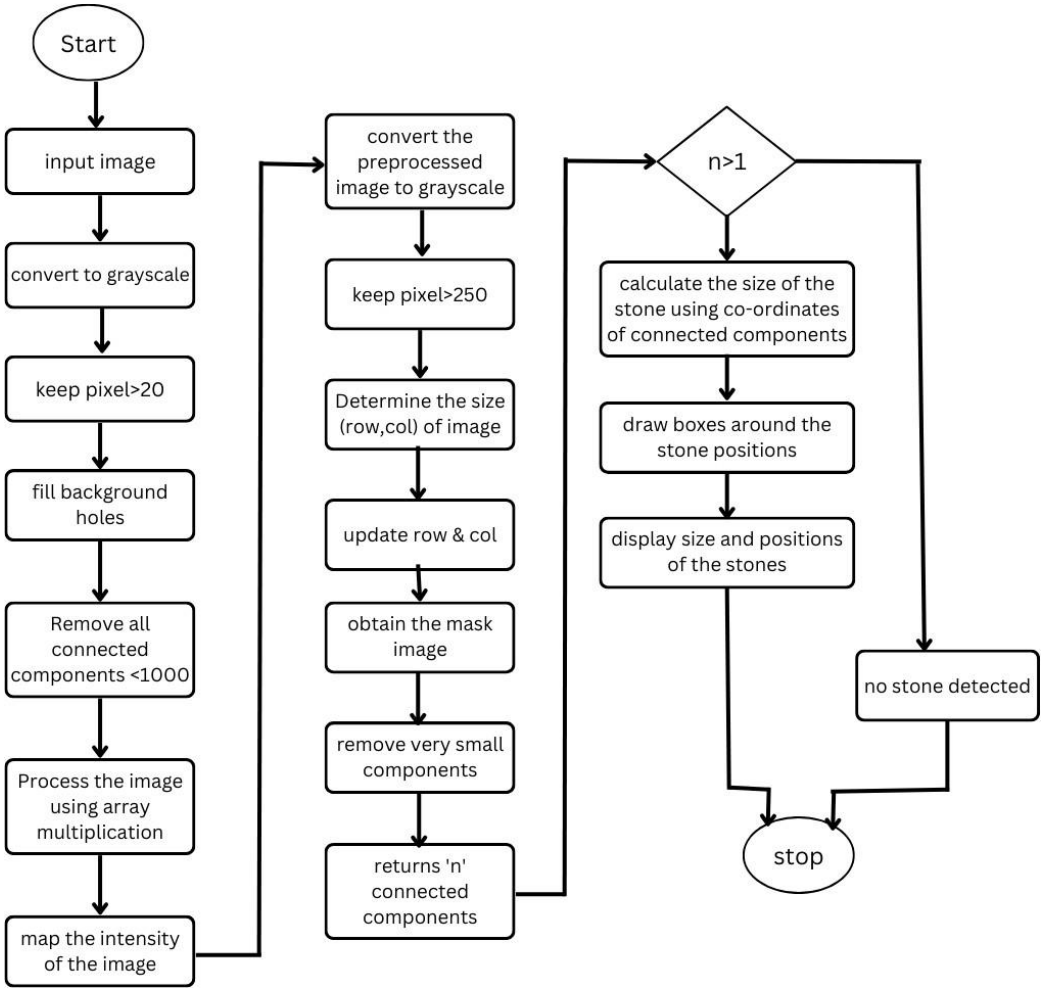


Fig. 4.6.1 Flow chart of the system

IMPLEMENTATION

5. IMPLEMENTATION

The objective of the coding or programming phase is to translate the design of the system produced during the design phase into code in a given programming language, which can be executed by a computer and that performs the computation specified by the design.

The coding phase affects both testing and maintenance. The goal of coding is not to reduce the implementation cost, but the goal should be to reduce the cost of later phases.

5.1 Coding Approach

There are two major approaches for coding any software system. They are Top-Down approach and bottom up approach.

Bottom-up Approach can best suit for developing the object-oriented systems. During system design phase, we decompose the system into an appropriate number of subsystems, for which objects can be modelled independently. These objects exhibit the way the subsystems perform their operations.

Once objects have been modelled, they are implemented by means of coding. Even though related to the same system as the objects are implemented of each other, the Bottom-Up approach is more suitable for coding these objects. In this approach, we first do the coding of objects independently and then we integrate these modules into one system to which they belong.

This code will first allow the admin to login with a valid user id and password. After this, admin will select the admission or academic module and enter all the students' details branch wise. TPO will select placement module and enter the details based on academic performance. Reports are generated accordingly and can be viewed by teaching staff and students.

5.2 Information Handling

Any software system requires some amount of information during its operation selection of appropriate data structures can help us to produce the code so that objects of the system can better operate with the available information decreased complexity.

5.3 Programming Style

Programming style deals with act of rules that a programmer must follow so that the characteristics of coding such as Traceability, Understandability, Modifiability, and Extensibility can be satisfied. In this current system, we followed the coding rules for naming the variables and methods.

5.4 Verification and Validation

Verification is the process of checking the product built is right. Validation is the process of checking whether the right product is built. During the Development of the system, Coding for the object has been thoroughly verified from different aspects regarding their design, in the way they are integrated etc. The various techniques that have been followed for validation discussed in testing the current system.

Validations applied to the entire system at two levels:

Form level Validation:

Validations of all the inputs given to the system at various points in the forms are validated while navigating to the next form. System raises appropriate custom and pre-defined exceptions to alert the user about the errors occurred or likely to occur.

Field level Validation:

Validations at the level of individual controls are also applied wherever necessary. System pops up appropriate and sensuous dialogs wherever necessary. In this project**,** validations are performed on each individual control. If any one of text field is not filled or any wrong, click occurs then system will generate appropriate exceptions.

5.5 Implementation of Project

Kidney Stone Detection

The implementation of the project involves the utilization of these methods.

rgb2gray

In MATLAB, the ‘rgb2gray’ function is used to convert an RGB (Red-Green-Blue) colour image to a grayscale image. The function takes an input image in the form of an M-by-N-by-3 array, where M and N are the dimensions of the image and 3 represents

the RGB colour channels. The output of the function is an M-by-N grayscale image that represents the luminance of the original image. The ‘rgb2gray’ function calculates the luminance of each pixel in the input image by combining the values of the red, green, and blue channels using the formula:

$$L = 0.2989 * R + 0.5870 * G + 0.1140 * B$$

where R, G, and B represent the values of the red, green, and blue channels, respectively. The resulting grayscale image has a single intensity value for each pixel, which represents the brightness of that pixel. The ‘rgb2gray’ function is commonly used in Image processing tasks where colour information is not important and only the brightness or intensity of the image is relevant, such as in edge detection or object recognition.

Thresholding

In MATLAB, thresholding is a technique used to segment images into binary images by separating the image pixels into two classes based on their intensities. The function used to perform thresholding in MATLAB is the ‘imbinarize’ function. The ‘imbinarize’ function takes an input grayscale image and a threshold value as arguments, and returns a binary image where pixels with intensities below the threshold are set to 0 (black) and pixels with intensities above the threshold are set to 1 (white). There are different methods of thresholding that can be used in MATLAB, including:

1. Global thresholding: A single threshold value is used for the entire image.

2. Adaptive thresholding: The threshold value is calculated locally for each pixel based on the intensity of its neighbouring pixels.

3. Otsu's thresholding: A threshold value is calculated automatically by minimizing the intra-class variance of the intensities in the image.

Thresholding is a common technique used in Image processing applications such as object detection, edge detection, and image segmentation. By converting the image into a binary image, it becomes easier to isolate objects or regions of interest, which can be further processed or analysed.

Medfilt2

In MATLAB, medfilt2 is a function that performs median filtering on a 2D matrix or an image. Median filtering is a technique used to remove noise from an image or a signal. It replaces each pixel in the image with the median value of its neighbouring pixels. The median value is the value that separates the higher half and the lower half of a set of values. The medfilt2 function takes two arguments: the input matrix or image, and the size of the filter window. The filter window is a matrix that defines the neighbourhood of each pixel. The size of the filter window determines the number of neighbouring pixels that are considered in the calculation of the median value.

Here is the syntax of the medfilt2 function:

```css
B = medfilt2(A, [M N])
```

where A is the input matrix or image, [M N] is the size of the filter window, and B is the output matrix or image.

The medfilt2 function performs median filtering on the input matrix or image, using a filter window of size M-by-N. It returns the filtered output matrix or image, where each pixel is replaced by the median value of its neighbouring pixels. The medfilt2 function is commonly used in Image processing applications to remove noise from images. It can be used to improve the quality of images by reducing the effect of random variations in pixel intensity.

Imfill

'imfill' is a function in MATLAB that fills the interior of objects in a binary image. It can also fill the holes in the binary image, which can be useful in Image processing applications.

The basic syntax of the imfill function is:

```scss
BW2 = imfill(BW, 'holes')
```

Here, 'BW' is the binary image that needs to be filled, and 'holes' is the option that tells the function to fill the holes in the image. 'BW2' is the output image with the holes filled. The 'imfill' function works by identifying the background pixels that are not connected to the image border and then filling them with the foreground value. The algorithm can handle binary images with multiple objects, and it can fill holes in each object separately. In addition to the 'holes' option, the 'imfill' function also has other options, such as 'holes', 'boundaries', and 'objects'. These options allow you to fill only the interior or exterior of objects, or only the boundaries of objects.

The 'imfill' function can be useful in a variety of Image processing applications, such as medical image analysis, object recognition, and feature extraction. It can help to improve the accuracy of image segmentation and reduce the amount of manual processing required.

bwareaopen

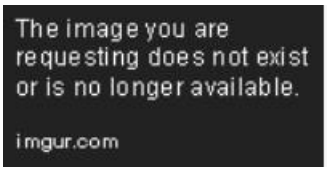
'bwareaopen' is a function in MATLAB's Image processing Toolbox that removes small connected components (objects) from a binary image. The function takes two input arguments: the binary image to be processed and a threshold area value. Any connected component (object) in the binary image whose area is less than or equal to the threshold value is removed.

The syntax for using 'bwareaopen' is as follows:

```scss
outputImage = bwareaopen(inputImage, thresholdArea)
```

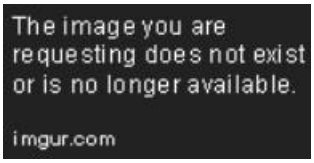
Here, 'inputImage' is the binary image to be processed, and 'thresholdArea' is the minimum area of connected components to be retained in the output image. The function returns the processed binary image as 'outputImage'. In other words, 'bwareaopen' can be used to remove small objects that may appear as noise or artifacts in an image. It is particularly useful for preprocessing images before performing more complex operations such as segmentation, feature extraction, or object recognition.

For example, consider the following binary image:



Using 'bwareaopen' with a threshold area of 50 pixels, we can remove the small objects and obtain a cleaner binary image as shown below:

```scss
bwClean = bwareaopen(bwNoisy, 50);
```



As you can see, the small objects in the original image have been removed, resulting in a clearer and more meaningful binary image.

repmat

The 'repmat' function in MATLAB is used to replicate and tile an array into a larger array. It is a shorthand for "repeat matrix".

The syntax for 'repmat' is as follows:

```css
B = repmat(A, m, n)
```

where 'A' is the matrix to be replicated, and 'm' and 'n' are the number of times that 'A' should be repeated in the row and column directions, respectively.

For example, if 'A' is a 2-by-3 matrix and 'm' is 2 and 'n' is 3, then ‘repmat(A, m, n)’ will create a 4-by-9 matrix by replicating ‘A’ two times in the row direction and three times in the column direction:

```
A = [1 2 3; 4 5 6];
B = repmat(A, 2, 3);

B =
     1     2     3     1     2     3     1     2     3
     4     5     6     4     5     6     4     5     6
     1     2     3     1     2     3     1     2     3
     4     5     6     4     5     6     4     5     6
```

Note that 'repmat' does not actually create copies of the input array 'A'. Instead, it creates a new array 'B' that consists of 'm' copies of 'A' in the row direction and 'n' copies of 'A' in the column direction. 'repmat' can be useful in a variety of situations, such as when you need to create a larger matrix for data manipulation or when you need to repeat a pattern of values in a matrix.

Imadjust

In MATLAB, 'imadjust' is a function used to adjust the intensity values of an image. It allows you to perform contrast adjustments, grayscale transformations, and colour mapping. The 'imadjust' function takes an input image and maps its intensity values to a new range of values. The new range can be specified using a variety of methods, including specifying the minimum and maximum values of the output range, specifying the percentile values of the input range, and using a gamma correction.

Here is the general syntax for using 'imadjust' :

```
J = imadjust(I, [low_in high_in], [low_out high_out], gamma)
```

Where 'I' is the input image, '[low_in high_in]' is the range of intensity values in the input image to be mapped to the output range, '[low_out high_out]' is the output range of intensity values, and 'gamma' is the gamma value used for gamma correction.

Here are some common use cases for 'imadjust':

- Adjusting contrast: If an image has low contrast, you can use 'imadjust' to increase the contrast by specifying the minimum and maximum values of the

output range. For example, you can set '[0.1 0.9]' as the output range to stretch the intensity values from 10% to 90% of the input range.

- Grayscale transformation: You can use 'imadjust' to transform an RGB image to grayscale by specifying a single output value range. For example, you can set '[0 1]' as the output range to map all intensity values to grayscale range of 0 to1.
- Colour mapping: You can use 'imadjust' to map the intensity values of an RGB image to a new colour range. For example, you can set '[0.3 0.7]' as the output range to map the intensity values to a range of colours between blue and yellow.

Overall, 'imadjust' is a powerful tool for adjusting the intensity values of an image and can be used to enhance the visual quality of images for various applications.

Size()

In Matlab, the'size()'function is used to determine the size of a matrix or an array. It returns a vector containing the number of rows and columns (and possibly other dimensions) of the input argument.

The syntax for the 'size()' function is as follows:

```scss
sz = size(A)
```

Where 'A' is the input matrix or array, and 'sz' is a vector containing the dimensions of 'A'.

Here are some examples of using the 'size()' function in Matlab:

```css
A = [1 2 3; 4 5 6; 7 8 9];  % create a 3x3 matrix
sz = size(A);               % returns sz = [3 3], the dimensions of A
num_rows = sz(1);           % num_rows = 3, the number of rows in A
num_cols = sz(2);           % num_cols = 3, the number of columns in A
```

```scss
B = rand(2, 3, 4);          % create a 2x3x4 array of random numbers
sz = size(B);               % returns sz = [2 3 4], the dimensions of B
num_rows = sz(1);           % num_rows = 2, the number of rows in the first dimensio
num_cols = sz(2);           % num_cols = 3, the number of columns in the second dime
num_third_dim = sz(3);      % num_third_dim = 4, the number of elements in the third
```

In summary, the 'size()' function is a useful tool for determining the dimensions of matrices and arrays in Matlab, which can be used for many purposes including indexing, reshaping, and other operations.

roipoly

In MATLAB, 'roipoly' is a function that allows the user to define a polygonal region of interest (ROI) on an image. The function creates a binary mask, which can be used to extract or modify the image pixels within the defined region.

Here's how 'roipoly' works:

The user selects a polygonal ROI by clicking on the image to define the vertices of the polygon. The ROI can be any shape, including concave or convex polygons. Once the ROI is defined, 'roipoly' creates a binary mask with the same size as the image, where the pixels inside the ROI are set to 1 and the pixels outside the ROI are set to 0.The user can use this binary mask to extract or modify the pixels within the ROI. For example, the user can use the mask to compute the mean, median, or standard deviation of the pixel values inside the ROI, or to apply a filter to only the pixels inside the ROI. The user can also modify the image by replacing the pixel values inside the ROI with new values. For example, the user can draw a line inside the ROI or fill it with a new colour. 'roipoly' is particularly useful in Image processing and analysis tasks where the user needs to isolate a specific region of interest in the image and apply a specific operation to it.

Bwlabel

The bwlabel function in MATLAB is used to label the connected components in a binary image. It takes a binary image as input and outputs a matrix with the same size as the input image, where each pixel is labelled with an integer that corresponds to the connected component to which it belongs.

The syntax for using bwlabel is as follows:

```scss
[L, num] = bwlabel(BW, n)
```

where BW is the binary image, n is the connectivity (which can be 4 or 8), L is the output matrix with the labels, and num is the total number of connected components found in the image. The bwlabel function works by scanning the image from left to right and from top to bottom, labelling each pixel based on the connectivity specified. The labelling is done by assigning a unique integer value to each connected component found in the image. Once the labelling is done, the function returns the output matrix L, where each pixel is assigned a label corresponding to the connected component to which it belongs. The num output parameter gives the total number of connected components found in the image. bwlabel is a very useful function in Image processing applications where the identification of connected components is necessary for further analysis or processing.

The proposed method for detecting kidney stones is divided into 2 phases as:

- Pre-processing
- Detection of Kidney stones

The phase A, Pre-processing has Gray scaling, Thresholding and Median Filtering. Whereas the phase B, Detection of Kidney Stones deals with Creating a line around the stone, Identifying connected components, calculating the size of detected stones and plotting boxes around the detected stones. The detailed description of all phases is presented below.

A. Pre-processing of the US scan image:

1. **Converting the image to grayscale**: The original image is converted to grayscale to reduce the amount of information and simplify the processing steps. This is done using the function "rgb2gray()".
2. **Thresholding the image:** The thresholding process is applied to the grayscale image to simplify it further. A threshold level of 20 is applied using the expression "b>20". This process segments the image into black and white pixels.

3. **Applying median filtering:** Median filtering is applied to the image to reduce the noise present in it. This helps to preserve the details in the image. The median filter size used is [5 5] and the function "medfilt2()" is used to apply median filtering.
4. **Thresholding the image again**: After median filtering, the image is thresholded again to further simplify it. A threshold level of 250 is applied using the expression "mo>250".

B. Detection of kidney stones:

Creating a line around the stone: A line is created around the estimated location of the kidney stone using the "roipoly()" function. The line is created by specifying the coordinates of its endpoints using the "row" and "col" arrays.

Identifying connected components (stones): The function "bwlabel()" is used to identify the connected components (stones) in the image. The output of this function is the label matrix, which contains the information of the different stones in the image.

Calculating the size of each stone: The size of each stone is calculated by finding the number of pixels belonging to each stone and converting it to the equivalent area in mm^2. The size of the stone is then calculated as the square root of the area in mm^2.

Plotting rectangles around the stones: A rectangle is plotted around each stone using the "rectangle()" function. The position of each rectangle is specified using the "rectPositions" array, which is calculated based on the positions of the pixels belonging to each stone. The colour of the rectangles and their width are also specified to distinguish them from the background.

5.5.1 Algorithmic Steps

Step 1: Read the image file through the "uigetfile" function.

Step 2: Convert the image to grayscale using "rgb2gray" function.

Step 3: Apply thresholding to the grayscale image using the "b>20" condition and fill the holes in the image using "imfill" function.

Step 4: Remove small objects from the image using "bwareaopen" function.

Step 5: Apply median filtering to the image using "medfilt2" function.

Step 6: Threshold the filtered image again using the "mo>250" condition.

Step 7: Get the dimensions of the image using the "size" function.

Step 8: Create a line around the stone and select the ROI using "roipoly" function.

Step 9: Label all connected components in the ROI using "bwlabel" function.

Step 10: For each connected component (stone), calculate its size by summing the area and taking the square root of the result. Store the size in the "stoneSizes" array.

Step 11: Draw rectangles around the stones using "rectangle" function and store the positions of the rectangles in the "rectPositions" array.

Step 12: If stones are detected, display the image with rectangles and provide the size of each stone. If no stones are detected, display a message indicating the absence of stones.

The below images represent the results of filters applied:

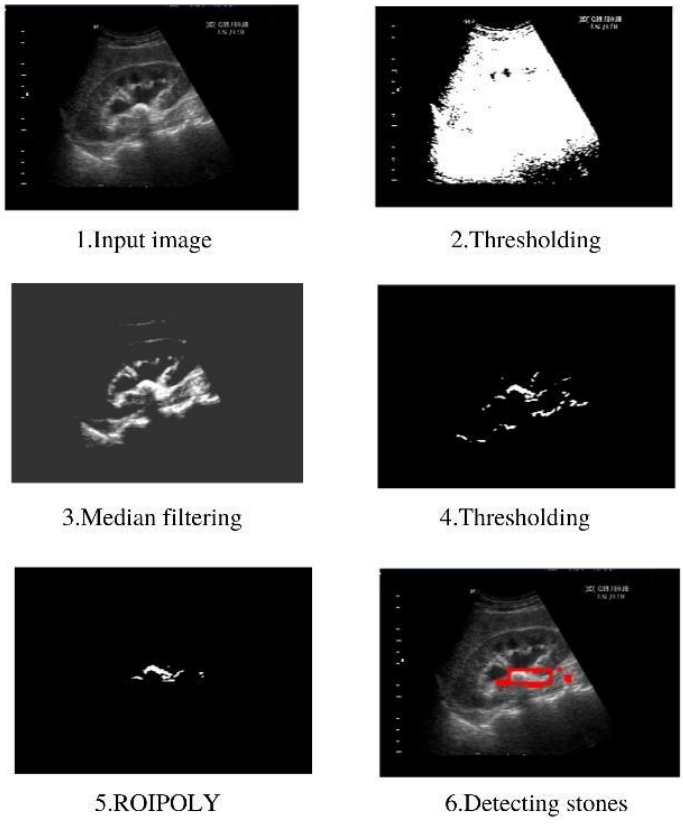


Fig.5.5.1 Results after Image processing filters applied.

User Interface

The process of pre-processing the image and detecting stone from the reconstructed image is all integrated with a graphical user interface developed using app designing tool provided in MATLAB to abstract the complex IDE of MATLAB to common users.

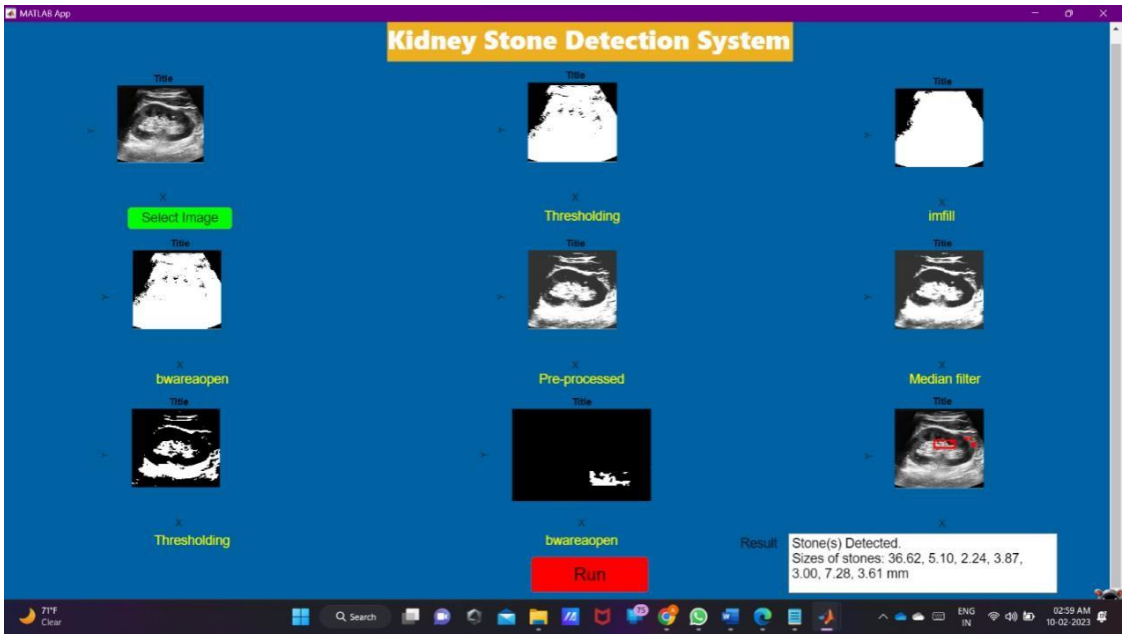


Fig.5.5.2 GUI of the system.

TESTING

6. TESTING

Testing is the process of finding differences between the expected behaviour specified by system models and the observed behaviour of the system. Testing is a critical role in quality assurance and ensuring the reliability of development and these errors will be reflected in the code, so the application should be thoroughly tested and validated.

Unit testing finds the differences between the object design model and its corresponding components. Structural testing finds differences between the system design model and a subset of integrated subsystems. Functional testing finds differences between the use case model and the system.

Finally, performance testing, finds differences between non-functional requirements and actual system performance. From modelling point of view, testing is the attempt of falsification of the system with respect to the system models. The goal of testing is to design tests that exercise defects in the system and to reveal problems.

6.1 Testing Activities

Testing a large system is a complex activity and like any complex activity. It has to be broke into smaller activities. Thus, incremental testing was performed on the project i.e., components and subsystems of the system were tested separately before integrating them to form the subsystem for system testing.

6.2 Testing Types

Unit Testing

Unit testing focuses on the building blocks of the software system that is the objects and subsystems. There are three motivations behind focusing on components. First unit testing reduces the complexity of overall test activities allowing focus on smaller units of the system, second unit testing makes it easier to pinpoint and correct faults given that few components are involved in the rest. Third unit testing allows parallelism in the testing activities, that is each component are involved in the test. Third unit testing allows parallelism in the testing activities that is each component can be tested independently of one another. The following are some unit testing techniques.

- **Equivalence testing**:

  It is a black box testing technique that minimizes the number of test cases. The possible inputs are partitioned into equivalence classes and a test case is selected for each class.

- **Boundary testing:**

  It is a special case of equivalence testing and focuses on the conditions at the boundary of the equivalence classes. Boundary testing requires that the elements be selected from the edges of the equivalence classes.

- **Path testing:**

  It is a white box testing technique that identifies faults in the implementation of the component the assumption here is that exercising all possible paths through the code at least once. Most faults will trigger failure. This acquires knowledge of source code.

Integrating Testing

Integration testing defects faults that have not been detected. During unit testing by focusing on small groups on components two or more components are integrated and tested and once tests do not reveal any new faults, additional components are added to the group. This procedure allows testing of increasing more complex parts on the system while keeping the location of potential faults relatively small. I have used the following approach to implements and integrated testing.

Top-down testing strategy unit tests the components of the top layer and then integrated the components of the next layer down. When all components of the new layer have been tested together, the next layer is selected. This was repeated until all layers are combined and involved in the test.

Validation Testing

The systems completely assembled as package, the interfacing have been uncovered and corrected, and a final series of software tests are validation testing. The validation testing is nothing but validation success when system functions in a manner that can be reasonably expected by the customer. The system validation had done by series of Black-box test methods.

System Testing

1. System testing ensures that the complete system compiles with the functional requirements and non-functional requirements of the system, the following are some system testing activities.
2. Functional testing finds differences between the functional between the functional requirements and the system. This is a black box testing technique. Test cases are divided from the use case model.
3. Performance testing finds differences between the design and the system the design goals are derived from the functional requirements.
4. Pilot testing the system is installed and used by a selected set of users – users exercise the system as if it had been permanently installed.
5. Acceptance testing, I have followed benchmarks testing in a benchmark testing the client prepares a set of test cases represent typical conditions under which the system operates. In our project, there are no existing benchmarks.

Installation testing, the system is installed in the target environment.

Black Box Testing also known as Behavioural Testing, is a software testing method in which the internal structure/design/implementation of the item being tested is not known to the tester. These tests can be functional or non-functional, though usually functional. This method is named so because the software program, in the eyes of the tester, is like a black box; inside which one cannot see. This method attempts to find errors in the following categories:

- Incorrect or missing functions
- Interface errors
- Errors in data structures or external database access
- Behaviour or performance errors
- Initialization and termination errors

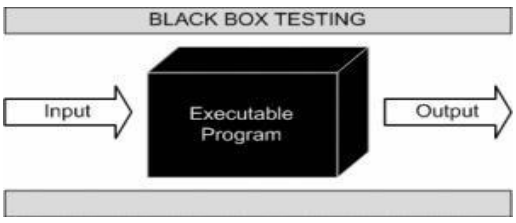


Fig. 6.2.1 Black Box Testing

6.3 Test cases

Test Case#	Test Case Description	Expected Result	Actual Result	Pass/Fail
1	US scan of kidney with no stones	No stone(s) detected	No stone(s) detected	PASS
2	US scan of kidney with stones	Stone(s) is/are Detected Size of stone 1: 10.7703mm Size of stone 2: 30.8383mm Size of stone 3: 11.2694mm Size of stone 4: 4.2426mm Size of stone 5: 2.2361mm Size of stone 6: 6.9282mm Size of stone 7: 3.1623mm	Stone(s) is/are Detected Size of stone 1: 10.7703mm Size of stone 2: 30.8383mm Size of stone 3: 11.2694mm Size of stone 4: 4.2426mm Size of stone 5: 2.2361mm Size of stone 6: 6.9282mm Size of stone 7: 3.1623mm	PASS
3	US scan of kidney with stones	Stone(s) is/are Detected Size of stone 1: 2.4495mm Size of stone 2: 7.4162mm Size of stone 3: 14.4914mm	Stone(s) is/are Detected Size of stone 1: 2.4495mm Size of stone 2: 7.4162mm Size of stone 3: 14.4914mm	PASS
4	US scan of kidney with no stones	No stone(s) detected	No stone(s) detected	PASS
5	US scan of kidney with stones	Stone(s) is/are Detected Size of stone 1: 4.899mm Size of stone 2: 5.7446mm	Stone(s) is/are Detected Size of stone 1: 4.899mm Size of stone 2: 5.7446mm	PASS
6	US scan of kidney with stones	Stone(s) is/are Detected Size of stone 1: 10.6301mm	Stone(s) is/are Detected Size of stone 1: 10.6301mm	PASS

		Size of stone 2: 2mm	Size of stone 2: 2mm	
7	US scan of kidney with stones	Stone(s) is/are Detected Size of stone 3: 7.4833mm Size of stone 4: 2.2361mm Size of stone 5: 4.3589mm	Stone(s) is/are Detected Size of stone 3: 7.4833mm Size of stone 4: 2.2361mm Size of stone 5: 4.3589mm	PASS
8	US scan of kidney with no stones	Stone(s) is/are Detected Size of stone 1: 2.4495mm Size of stone 2: 2.6458mm	Stone(s) is/are Detected Size of stone 1: 2.4495mm Size of stone 2: 2.6458mm	PASS
9	US scan of kidney with stones	Stone(s) is/are Detected Size of stone 1: 8.775mm Size of stone 2: 20.4695mm Size of stone 3: 4.2426mm	Stone(s) is/are Detected Size of stone 1: 8.775mm Size of stone 2: 20.4695mm Size of stone 3: 4.2426mm	PASS
10	US scan of kidney with stones	Stone(s) is/are Detected Size of stone 1: 9.3808mm Size of stone 2: 2.4495mm	Stone(s) is/are Detected Size of stone 1: 9.3808mm Size of stone 2: 2.4495mm	PASS

RESULTS

7. RESULTS

7.1 Start Window

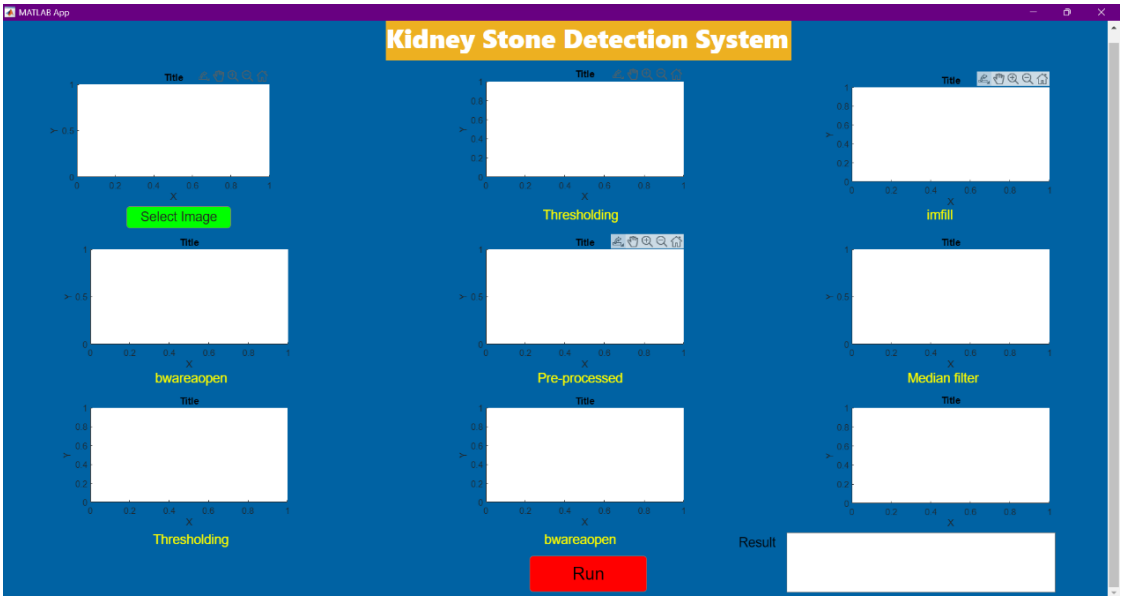


Fig. 7.1.1 Start Window

7.2 Image Selected with Stone

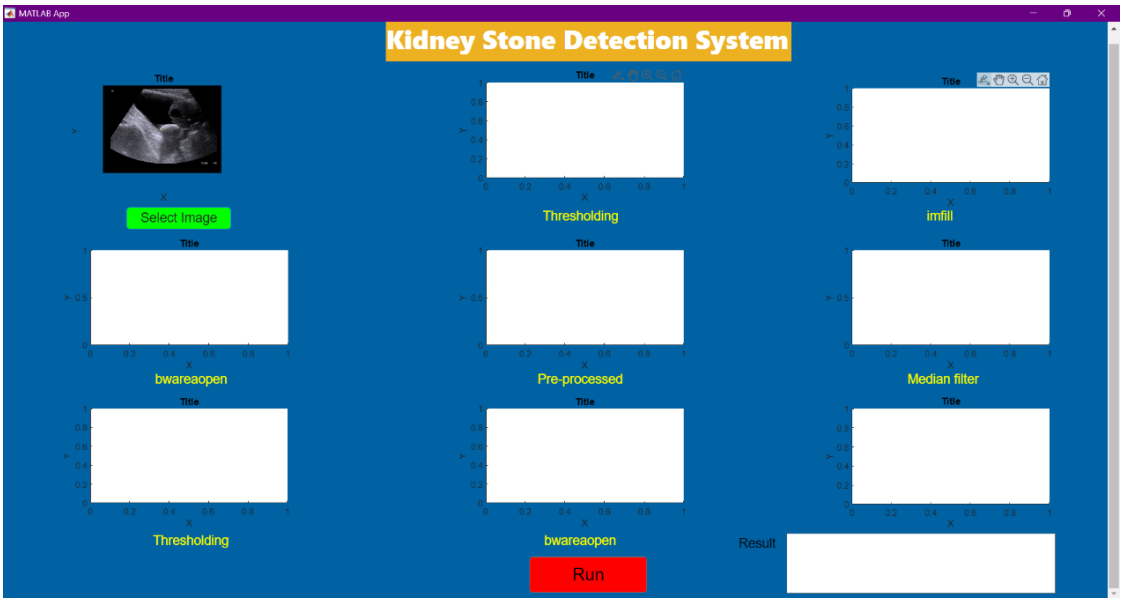


Fig. 7.2.1 image selected with stone

7.3 Stone Detection

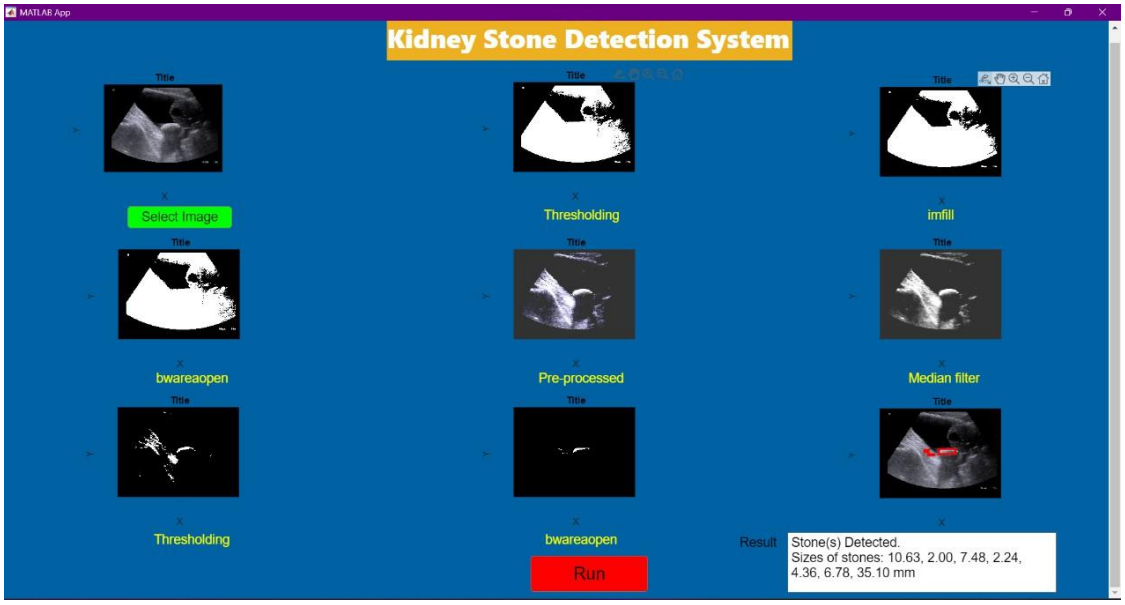


Fig. 7.3.1 Stone Detected

7.4 Sizes of detected stones



Fig. 7.4.1 Sizes of detected stones

7.5 Image Selected without Stone

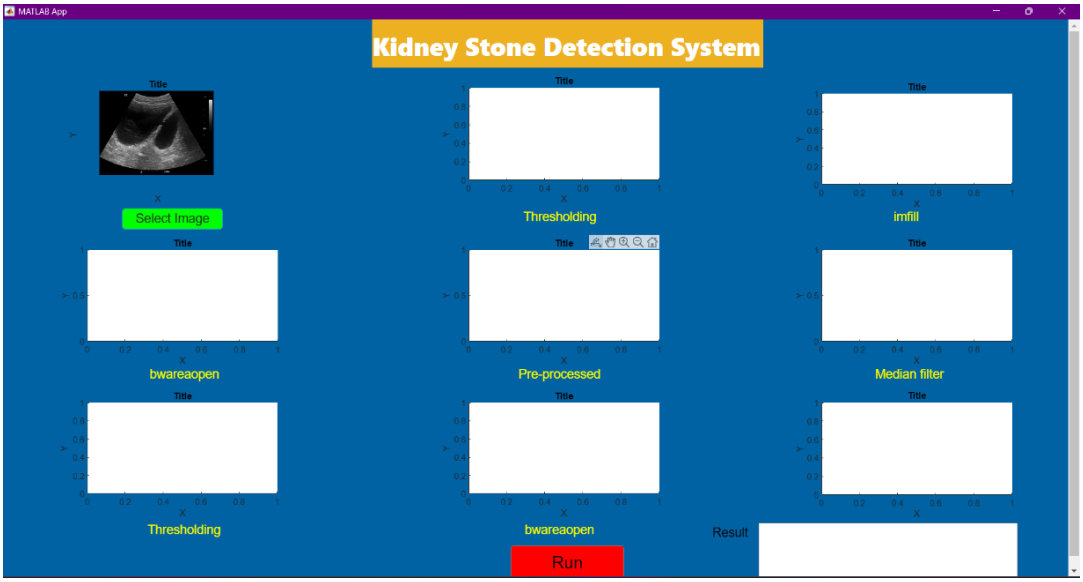


Fig. 7.5.1 Image Selected without Stone

7.6 Stone Detection

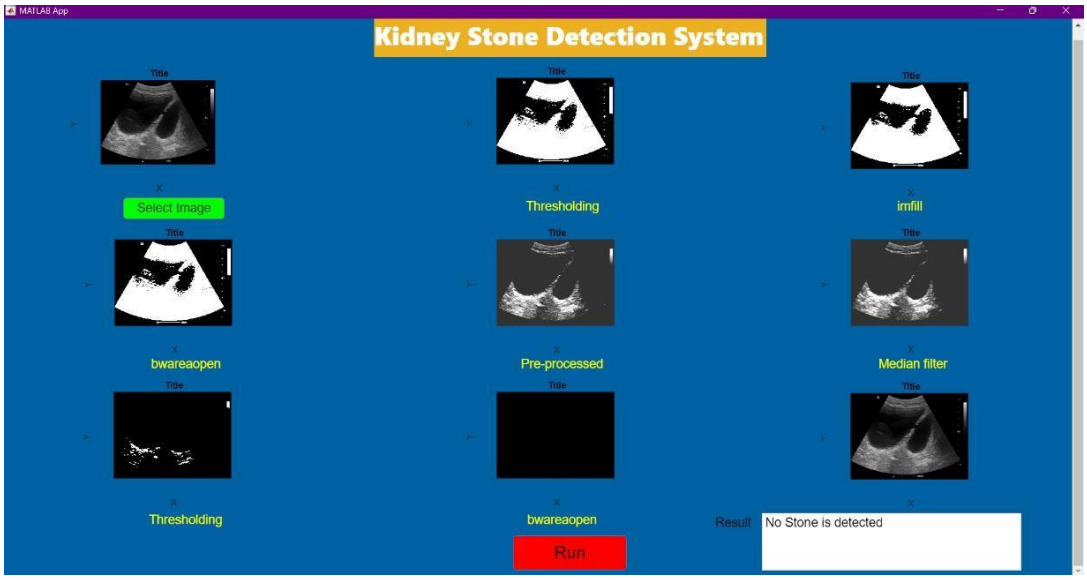


Fig. 7.6.1 Sizes of detected stones

7.7 Sizes of detected stones

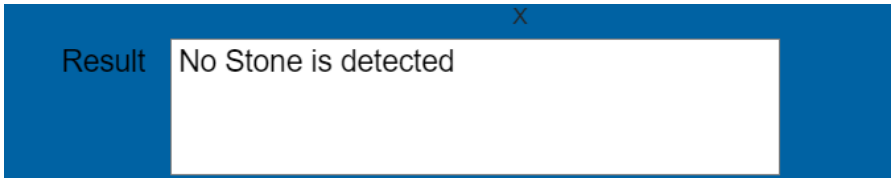


Fig. 7.7.1 Sizes of detected stones

SAMPLE CODE

8. SAMPLE CODE

8.1 Code for Application

```
clc

clear all

close all


% Read the image file

[filename, pathname]=uigetfile('*.*', 'Pick a MATLAB code file');

filename=strcat(pathname,filename);

a=imread(filename);

imshow(a);


% Convert the image to grayscale

b=rgb2gray(a);


% Apply thresholding to the image

c=b>20;

imshow(c);

d=imfill(c,'holes');

imshow(d);

e=bwareaopen(d,1000);

imshow(e);

PreprocessedImage=uint8(double(a).*repmat(e,[1 1 3]));

PreprocessedImage=imadjust(PreprocessedImage,[0.3 0.7],[])+50;
```

```
imshow(PreprocessedImage);


% Apply median filtering
uo=rgb2gray(PreprocessedImage);
mo=medfilt2(uo,[5 5]);
imshow(mo);
% Threshold the image again
po=mo>250;
imshow(po);
% Get the dimensions of the image
[r,c]=size(po);


% Create the line around the stone
x1=r/2;
y1=c/3;
row=[x1 x1+200 x1+200 x1];
col=[y1 y1 y1+40 y1+40];
BW=roipoly(po,row,col);
k=po.*double(BW);
M=bwareaopen(k,4);
imshow(M)
[L, num]=bwlabel(M);
```

```
% Initialize arrays to store the information of stones
stoneSizes = [];
rectPositions = [];

% Check if stone is detected
if(num >= 1)
    disp('Stone(s) is/are Detected');

    % Loop through all connected components (stones)
    for i = 1:num
        % Calculate the size of the stone
        [r, c]=find(L==i);
        stoneArea = sum(sum(L==i)) * (1^2); % in mm^2
        stoneSize = sqrt(stoneArea); % in mm
        stoneSizes = [stoneSizes, stoneSize];

        % Draw a rectangle around the stone
        rectPos = [min(c), min(r), max(c) - min(c) + 1, max(r) - min(r) + 1];
        rectPositions = [rectPositions; rectPos];

        disp(['Size of stone ', num2str(i), ': ', num2str(stoneSize), 'mm']);
    end
```

```
    % Plot the image with rectangles around the stones
    imshow(a);
    hold on
    for i = 1:num
        rectangle('Position', rectPositions(i,:), 'EdgeColor', 'r', 'LineWidth', 2);
    end
    hold off
else
    imshow(a);
    disp('No Stone is detected');
end
```

8.2 Code for GUI

```
classdef app2 < matlab.apps.AppBase
    % Properties that correspond to app components
    properties (Access = public)
        UIFigure              matlab.ui.Figure
        bwareaopenLabel_2     matlab.ui.control.Label
        ThresholdingLabel_6   matlab.ui.control.Label
        MedianfilterLabel     matlab.ui.control.Label
        PreprocessedLabel     matlab.ui.control.Label
        bwareaopenLabel       matlab.ui.control.Label
        imfillLabel           matlab.ui.control.Label
        ThresholdingLabel     matlab.ui.control.Label
        KidneyStoneDetectionSystemLabel  matlab.ui.control.Label
```

```
        TextArea              matlab.ui.control.TextArea
        ResultTextAreaLabel   matlab.ui.control.Label
        RunButton             matlab.ui.control.Button
        SelectImageButton     matlab.ui.control.Button
        UIAxes6               matlab.ui.control.UIAxes
        UIAxes9               matlab.ui.control.UIAxes
        UIAxes8               matlab.ui.control.UIAxes
        UIAxes7               matlab.ui.control.UIAxes
        UIAxes5               matlab.ui.control.UIAxes
        UIAxes4               matlab.ui.control.UIAxes
        UIAxes3               matlab.ui.control.UIAxes
        UIAxes2               matlab.ui.control.UIAxes
        axes1                 matlab.ui.control.UIAxes
    end
    % Callbacks that handle component events
    methods (Access = private)

        % Button pushed function: SelectImageButton
        function SelectImage_Callback(app, event)
            [filename, pathname] = uigetfile({'*.jpg;*.jpeg;*.png;*.bmp;*.gif','All Image
Files'});
            if filename ~= 0
                % Read the selected image
                image = imread(fullfile(pathname, filename));
```

```
            % Display the selected image in the first axes
            imshow(image, 'Parent', app.axes1);
        end
    end

    % Button pushed function: RunButton
    function runCodeButtonPushed(app, event)
            % Get the input image from the axes component
image = get(app.axes1, 'Children');
image = get(image, 'CData');

% Convert the image to grayscale
b = rgb2gray(image);

% Apply thresholding to the image
c = b > 20;
imshow(c, 'Parent', app.UIAxes3);
d = imfill(c, 'holes');
imshow(d, 'Parent', app.UIAxes4);
e = bwareaopen(d, 1000);
imshow(c, 'Parent', app.UIAxes5);
PreprocessedImage = uint8(double(image).*repmat(e, [1 1 3]));
PreprocessedImage = imadjust(PreprocessedImage, [0.3 0.7], []) + 50;
imshow(PreprocessedImage, 'Parent', app.UIAxes6);
```

```
% Apply median filtering
uo = rgb2gray(PreprocessedImage);
mo = medfilt2(uo, [5 5]);
imshow(mo, 'Parent', app.UIAxes7);
% Threshold the image again
po = mo > 250;
imshow(po, 'Parent', app.UIAxes8);
% Get the dimensions of the image
[r, c] = size(po);

% Create the line around the stone
x1 = r/2;
y1 = c/3;
row = [x1 x1+200 x1+200 x1];
col = [y1 y1 y1+40 y1+40];
BW = roipoly(po, row, col);
k = po.*double(BW);
M = bwareaopen(k, 4);
imshow(M, 'Parent', app.UIAxes9);
[L, num] = bwlabel(M);

% Initialize arrays to store the information of stones
stoneSizes = [];
rectPositions = [];
```

```
if(num >= 1)
    % Loop through all connected components (stones)
    for i = 1:num
        % Calculate the size of the stone
        [r, c] = find(L == i);
        stoneArea = sum(sum(L == i)) * (1^2); % in mm^2
        stoneSize = sqrt(stoneArea); % in mm
        stoneSizes = [stoneSizes, stoneSize];

        % Draw a rectangle around the stone
        rectPos = [min(c), min(r), max(c) - min(c) + 1, max(r) - min(r) + 1];
        rectPositions = [rectPositions; rectPos];
    end

    % Plot the image with rectangles around the stones
    imshow(image, 'Parent', app.UIAxes2);
    hold on
    for i = 1:num
        rectangle('Position', rectPositions(i,:), 'EdgeColor', 'r', 'LineWidth', 2,  'Parent',
app.UIAxes2);
    end
    hold off

    % Display the sizes of stones in the text box
    stoneSizesString = sprintf('%.2f, ', stoneSizes);
```

```
    app.TextArea.Value = ['Stone(s) Detected.',newline, 'Sizes of stones: ',
stoneSizesString(1:end-2), ' mm'];
else
    % Plot the original image if no stone is detected
    imshow(image, 'Parent', app.UIAxes2);
    app.TextArea.Value = 'No Stone is detected';
end
        end
    end

    % Component initialization
    methods (Access = private)

        % Create UIFigure and components
        function createComponents(app)

            % Create UIFigure and hide until all components are created
            app.UIFigure = uifigure('Visible', 'off');
            app.UIFigure.Color = [0 0.3843 0.6392];
            app.UIFigure.Position = [100 100 1477 1083];
            app.UIFigure.Name = 'MATLAB App';
            app.UIFigure.Scrollable = 'on';

            % Create axes1
            app.axes1 = uiaxes(app.UIFigure);
```

```
title(app.axes1, 'Title')

xlabel(app.axes1, 'X')

ylabel(app.axes1, 'Y')

zlabel(app.axes1, 'Z')

app.axes1.Position = [63 758 293 182];


% Create UIAxes2

app.UIAxes2 = uiaxes(app.UIFigure);

title(app.UIAxes2, 'Title')

xlabel(app.UIAxes2, 'X')

ylabel(app.UIAxes2, 'Y')

zlabel(app.UIAxes2, 'Z')

app.UIAxes2.Position = [1090 253 300 185];


% Create UIAxes3

app.UIAxes3 = uiaxes(app.UIFigure);

title(app.UIAxes3, 'Title')

xlabel(app.UIAxes3, 'X')

ylabel(app.UIAxes3, 'Y')

zlabel(app.UIAxes3, 'Z')

app.UIAxes3.Position = [606 758 300 185];


% Create UIAxes4

app.UIAxes4 = uiaxes(app.UIFigure);
```

```
title(app.UIAxes4, 'Title')
xlabel(app.UIAxes4, 'X')
ylabel(app.UIAxes4, 'Y')
zlabel(app.UIAxes4, 'Z')
app.UIAxes4.Position = [1090 747 300 185];

% Create UIAxes5
app.UIAxes5 = uiaxes(app.UIFigure);
title(app.UIAxes5, 'Title')
xlabel(app.UIAxes5, 'X')
ylabel(app.UIAxes5, 'Y')
zlabel(app.UIAxes5, 'Z')
app.UIAxes5.Position = [82 493 300 185];

% Create UIAxes7
app.UIAxes7 = uiaxes(app.UIFigure);
title(app.UIAxes7, 'Title')
xlabel(app.UIAxes7, 'X')
ylabel(app.UIAxes7, 'Y')
zlabel(app.UIAxes7, 'Z')
app.UIAxes7.Position = [1090 493 300 185];

% Create UIAxes8
app.UIAxes8 = uiaxes(app.UIFigure);
```

```
title(app.UIAxes8, 'Title')
xlabel(app.UIAxes8, 'X')
ylabel(app.UIAxes8, 'Y')
zlabel(app.UIAxes8, 'Z')
app.UIAxes8.Position = [81 254 300 185];

% Create UIAxes9
app.UIAxes9 = uiaxes(app.UIFigure);
title(app.UIAxes9, 'Title')
xlabel(app.UIAxes9, 'X')
ylabel(app.UIAxes9, 'Y')
zlabel(app.UIAxes9, 'Z')
app.UIAxes9.Position = [606 254 300 185];

% Create UIAxes6
app.UIAxes6 = uiaxes(app.UIFigure);
title(app.UIAxes6, 'Title')
xlabel(app.UIAxes6, 'X')
ylabel(app.UIAxes6, 'Y')
zlabel(app.UIAxes6, 'Z')
app.UIAxes6.Position = [606 493 300 185];

% Create SelectImageButton
app.SelectImageButton = uibutton(app.UIFigure, 'push');
```

```
            app.SelectImageButton.ButtonPushedFcn = createCallbackFcn(app,
@SelectImage_Callback, true);
            app.SelectImageButton.BackgroundColor = [0 1 0];
            app.SelectImageButton.FontSize = 18;
            app.SelectImageButton.FontColor = [0.149 0.149 0.149];
            app.SelectImageButton.Position = [161 719 144 30];
            app.SelectImageButton.Text = 'Select Image';

            % Create RunButton
            app.RunButton = uibutton(app.UIFigure, 'push');
            app.RunButton.ButtonPushedFcn = createCallbackFcn(app,
@runCodeButtonPushed, true);
            app.RunButton.BackgroundColor = [1 0 0];
            app.RunButton.FontSize = 24;
            app.RunButton.Position = [705 90 162 50];
            app.RunButton.Text = 'Run';

            % Create ResultTextAreaLabel
            app.ResultTextAreaLabel = uilabel(app.UIFigure);
            app.ResultTextAreaLabel.HorizontalAlignment = 'right';
            app.ResultTextAreaLabel.FontSize = 18;
            app.ResultTextAreaLabel.Position = [988 205 56 23];
            app.ResultTextAreaLabel.Text = 'Result';

            % Create TextArea
```

```
            app.TextArea = uitextarea(app.UIFigure);
            app.TextArea.FontSize = 18;
            app.TextArea.Position = [1059 112 330 118];

            % Create KidneyStoneDetectionSystemLabel
            app.KidneyStoneDetectionSystemLabel = uilabel(app.UIFigure);
            app.KidneyStoneDetectionSystemLabel.BackgroundColor = [0.9294 0.6902
0.1294];
            app.KidneyStoneDetectionSystemLabel.FontName = 'Segoe UI Black';
            app.KidneyStoneDetectionSystemLabel.FontSize = 36;
            app.KidneyStoneDetectionSystemLabel.FontColor = [1 1 1];
            app.KidneyStoneDetectionSystemLabel.Position = [508 969 557 69];
            app.KidneyStoneDetectionSystemLabel.Text = 'Kidney Stone Detection
System';

            % Create ThresholdingLabel
            app.ThresholdingLabel = uilabel(app.UIFigure);
            app.ThresholdingLabel.HorizontalAlignment = 'center';
            app.ThresholdingLabel.FontSize = 18;
            app.ThresholdingLabel.FontColor = [1 1 0];
            app.ThresholdingLabel.Position = [721 726 109 23];
            app.ThresholdingLabel.Text = 'Thresholding';

            % Create imfillLabel
            app.imfillLabel = uilabel(app.UIFigure);
```

```
app.imfillLabel.HorizontalAlignment = 'center';
app.imfillLabel.FontSize = 18;
app.imfillLabel.FontColor = [1 1 0];
app.imfillLabel.Position = [1249 726 41 23];
app.imfillLabel.Text = 'imfill';

% Create bwareaopenLabel
app.bwareaopenLabel = uilabel(app.UIFigure);
app.bwareaopenLabel.HorizontalAlignment = 'center';
app.bwareaopenLabel.FontSize = 18;
app.bwareaopenLabel.FontColor = [1 1 0];
app.bwareaopenLabel.Position = [198 471 104 23];
app.bwareaopenLabel.Text = 'bwareaopen';

% Create PreprocessedLabel
app.PreprocessedLabel = uilabel(app.UIFigure);
app.PreprocessedLabel.HorizontalAlignment = 'center';
app.PreprocessedLabel.FontSize = 18;
app.PreprocessedLabel.FontColor = [1 1 0];
app.PreprocessedLabel.Position = [714 471 122 23];
app.PreprocessedLabel.Text = 'Pre-processed';

% Create MedianfilterLabel
app.MedianfilterLabel = uilabel(app.UIFigure);
```

```
app.MedianfilterLabel.HorizontalAlignment = 'center';
app.MedianfilterLabel.FontSize = 18;
app.MedianfilterLabel.FontColor = [1 1 0];
app.MedianfilterLabel.Position = [1203 471 103 23];
app.MedianfilterLabel.Text = 'Median filter';

% Create ThresholdingLabel_6
app.ThresholdingLabel_6 = uilabel(app.UIFigure);
app.ThresholdingLabel_6.HorizontalAlignment = 'center';
app.ThresholdingLabel_6.FontSize = 18;
app.ThresholdingLabel_6.FontColor = [1 1 0];
app.ThresholdingLabel_6.Position = [195 227 109 23];
app.ThresholdingLabel_6.Text = 'Thresholding';

% Create bwareaopenLabel_2
app.bwareaopenLabel_2 = uilabel(app.UIFigure);
app.bwareaopenLabel_2.HorizontalAlignment = 'center';
app.bwareaopenLabel_2.FontSize = 18;
app.bwareaopenLabel_2.FontColor = [1 1 0];
app.bwareaopenLabel_2.Position = [723 227 104 23];
app.bwareaopenLabel_2.Text = 'bwareaopen';

% Show the figure after all components are created
app.UIFigure.Visible = 'on';
```

```
    end
end

% App creation and deletion
methods (Access = public)

    % Construct app
    function app = app2

        % Create UIFigure and components
        createComponents(app)

        % Register the app with App Designer
        registerApp(app, app.UIFigure)

        if nargout == 0
            clear app
        end
    end

    % Code that executes before app deletion
    function delete(app)

        % Delete UIFigure when app is deleted
```

```
            delete(app.UIFigure)
        end
    end
end
```

CONCLUSION

9. CONCLUSION

The Kidney Stone Detection System using Image processing in MatLab aims to detect stones present in a kidney using various Image processing techniques such as thresholding, medfilt2, roipoly, etc. The study found that the combination of these methods allowed for accurate detection of the stones, including their location and size. The results showed that the system was able to detect stones with high accuracy, making it a valuable tool for medical professionals. The Kidney Stone Detection System has a number of important implications and applications. Firstly, it can help medical professionals to diagnose kidney stones quickly and effectively, reducing the need for invasive procedures. While the Kidney Stone Detection System has shown promising results, there is still room for improvement. One suggestion for future work would be to evaluate the system on a larger dataset to ensure its accuracy and robustness. Additionally, the system could be integrated with other medical imaging modalities, such as ultrasound or CT scans, to provide a more comprehensive picture of the kidney and stone location. Further research could also be conducted to improve the system's performance and accuracy, making it an even more valuable tool for medical professionals.

REFERENCES

10. REFERENCES

[1] Tsao, Chang, and Lin in Ultrasonic Renal-Stone Detection and Identification for Extracorporeal Lithotripsy | IEEE Conference Publication | IEEE Xplore

[2] Viswanath and Gunasundari Design and analysis performance of kidney stone detection from ultrasound image by level set segmentation and ANN classification | IEEE Conference Publication | IEEE Xplore

[3] Angshuman Khan, Rupayan Das and M C Parameshwara https://iopscience.iop.org/article/10.1088/2631-8695/ac8b65

[4] Sakshi Garg, Angadpreet Walia, Abhilasha Singh & Anju Mishra A Study on Shape Detection: An Unexplored Parameter in the Gallstones Identification | SpringerLink

[5] A renal ultrasound is a very useful exam which allows doctors to quickly evaluate the condition of kidneys without exposing patients to radiation or using IV contract. https://ic.steadyhealth.com/ultrasound-procedure-for-kidney-stones

[6] F. L. Coe, A. Evan, and E. Worcester, "Kidney stone disease,"Journal of Clinical Investigation, vol. 115, no. 10, pp. 2598-2608,2019 https://www.jci.org/articles/view/26662/pdf.

[7] V. Romero, H. Akpinar, and D. G. Assimos, "Kidney stones: A global picture of prevalence, incidence, and associated risk factors," Reviews in Urology, vol. 12, no. 2-3, pp. 86-96, 2019 https://www.semanticscholar.org/paper/Digital-Image-Processing-Gonz%C3%A1lez-Woods/72ce0faa2d0be574f5cb88cfe6353a3ba40a08ae

[8] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 2nd ed., 2019, ch. 2, pp. 47-51 and ch. 10, pp. 568-611 https://www.semanticscholar.org/paper/Digital-Image-Processing-Gonz%C3%A1lez-Woods/72ce0faa2d0be574f5cb88cfe6353a3ba40a08ae.

[9] A. Rosenfeld and A. C. Kak, Digital Picture Processing, 2nd ed., Academic Press,2018,ch.10 https://www.sciencedirect.com/book/9780125973021/digital-picture-processing.

[10] Mehmet Baygin, Orhan Yaman, Prabal Datta Barua Exemplar Darknet19 feature generation technique for automated kidney stone detection with coronal CT images – ScienceDirect

[11] Kidney Stone Detection using Ultrasound Images by Tanya Borges, Akash Rai, Dharm Raj, Danish Ather, Keshav Gupta :: SSRN.

[12] A Gallery of High-Resolution, Ultrasound, Color Doppler & 3D Images (ultrasoundimages.com)